

ساخت MiniGPT مبتنی بر JAX

یک تمرین پنج‌قسمتی: از کاراکتر تا یک مدل کوچک GPT

ساخته‌شده با JAX و Flax · در دل استارت‌آپ‌های ایرانی، با یک شخصیت پیوسته به‌نام نیکا



این تمرین چیست؟



یک تمرین عملی پنج‌قسمتی که در آن از صفر یک مدل زبانی کوچک از خانواده‌ی GPT را با JAX و Flax می‌سازید و آموزش می‌دهید. تمرکز کامل روی معماری ترنسفورمر است و هر قسمت در یکی از استارت‌آپ‌های ایرانی روایت می‌شود؛ مهم‌تر از همه، قسمت‌ها پیوسته‌اند و خروجی هر قسمت ورودی قسمت بعدی است، تا در پایان از تبدیل ساده‌ی کاراکتر به عدد، به مدلی برسید که خودش متن تولید می‌کند.

از کجا شروع کنیم و چطور پیش برویم؟



به‌ترتیب از قسمت یک تا پنج پیش بروید. در هر قسمت ابتدا فایل `init.ipynb` را باز کنید، سلول‌ها را از بالا به پایین اجرا کنید، Learning note ها و نمودارها را با دقت بخوانید و هر جا `# TODO` دیدید آن را کامل کنید؛ اگر جایی گیر کردید، Learning note های همان قسمت و راهنمایی داخل همان سلول را دوباره بخوانید. توجه کنید که قسمت‌ها به هم وابسته‌اند، پس پیش از قسمت پنج حتماً قسمت چهار را اجرا کنید. در ادامه، داستان هر قسمت و کاری که انجام می‌دهید آمده است.

Snapp

۱ اسنپ – مبانی و توکن‌سازی

۱

داستان از اسنپ آغاز می‌شود، جایی که نیکا، مهندس تازه‌وارد تیم پلتفرم، مأمور می‌شود یک مدل زبانی را از صفر بسازد. در این قسمت ابتدا با JAX و سه ابزار کلیدی‌اش (`jit`، `grad` و `vmap`) آشنا می‌شوید، سپس یک `softmax` پایدار را با دست خودتان می‌نویسید و یک توکن‌ساز کاراکتری می‌سازید که متن را به عدد تبدیل می‌کند و در پایان داده را به شکل بچ‌های پیش‌بینی توکن بعدی آماده می‌کنید. خروجی این قسمت همان تنسورهای عددی است که پایه‌ی همه‌ی قسمت‌های بعدی‌اند.

Digikala

۲ دیجی‌کالا – مکانیزم توجه

۲

در دیجی‌کالا سراغ قلب ترنسفورمر می‌روید: مکانیزم توجه. یاد می‌گیرید هر توکن چگونه با ساختن بردارهای `Key`، `Value` و `Query` به بقیه «توجه» می‌کند، چرا امتیازها را بر ریشه‌ی بعد تقسیم می‌کنیم، و چگونه یک ماسک علی‌جلوی دیدن آینده را می‌گیرد؛ کدگذاری موقعیت هم اضافه می‌شود تا مدل ترتیب کلمات را بفهمد. خروجی این قسمت یک لایه‌ی `self-attention` است که می‌توانید نقشه‌ی توجه‌ش را ببینید و در قسمت بعد به‌کار بگیرید.

در تپسی، توجه تکسر را به توجه چندسر (multi-head) تعمیم می‌دهید تا چند «متخصص» موازی هرکدام الگوی متفاوتی یاد بگیرند، و این بار همه چیز را با ماژول‌های Flax مهندسی می‌کنید. سپس با افزودن شبکه‌ی MLP، نرمال‌سازی لایه‌ای و اتصال‌های باقی‌مانده، یک بلوک کامل ترنسفورمر می‌سازید. خروجی این قسمت همان آجری است که در قسمت بعد روی هم چیده می‌شود تا مدل ساخته شود.

روی ماشین کوئرا همه‌ی قطعات را کنار هم می‌گذارید: امبدینگ توکن و موقعیت، چند بلوک ترنسفورمر و یک لایه‌ی خروجی، تا MiniGPT ساخته شود. سپس آن را با زبان آنتروپی متقاطع، بهینه‌ساز AdamW و یک زمان‌بندی نرخ یادگیری آموزش می‌دهید و زبان را تماشا می‌کنید که از حدود ۴/۲ تا نزدیک ۱/۵ پایین می‌آید. خروجی این قسمت یک مدل آموزش‌دیده‌ی حدوداً ۲/۷ میلیون پارامتری است که در فایل minigpt_ckpt.pkl ذخیره می‌شود تا قسمت پنج آن را بخواند.

در دیوار، همان مدل آموزش‌دیده‌ی قسمت چهار را بارگذاری می‌کنید و وادارش می‌کنید متن بسازد. تولید خودبازگشتی را پیاده می‌کنید و سه اهرم کنترل تولید را می‌سازید: temperature برای میزان خلاقیت، و top-k و top-p برای مهار دنباله‌ی توکن‌های نامربوط؛ سپس با نگاهی به قوانین مقیاس می‌بینید فاصله‌ی این مدل کوچک تا GPT-2 و GPT-3 چقدر است. خروجی نهایی مدلی است که از یک نشانه‌ی شروع مثل «ROMEO:» متن تازه تولید می‌کند.

صورت مسئله

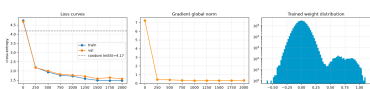


مأموریت شما این است که به‌جای نیکا یک MiniGPT را از صفر بسازید و آموزش دهید. در هر قسمت یک قطعه از مدل را پیاده‌سازی می‌کنید (توکن‌ساز، توجه، بلوک ترنسفورمر، حلقه‌ی آموزش و تولید) و چون قسمت‌ها پیوسته‌اند، خروجی هر قسمت ورودی قسمت بعد است. در پایان مدلی خواهید داشت که متن تازه تولید می‌کند. پیش‌نیاز: پایتون متوسط و جبر خطی مقدماتی؛ آشنایی مختصر با NumPy کافی است و JAX را در همین مسیر یاد می‌گیرید.

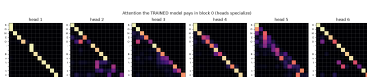
خروجی‌ها و کاربردها



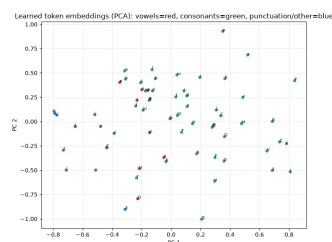
در طول تمرین چند نوع خروجی تولید می‌کنید و هرکدام هدفی روشن دارند. نمودارها – مانند منحنی زبان، نقشه‌ی توجه و فضای امبدینگ – برای فهم رفتار مدل و تأیید درستی کار شما هستند؛ با نگاه به آن‌ها می‌فهمید مدل واقعا در حال یادگیری است یا نه. فایل minigpt_ckpt.pkl وزن‌های مدل آموزش‌دیده را نگه می‌دارد تا قسمت بعد بدون آموزش دوباره از آن استفاده کند. و در نهایت، متن تولیدشده نشان می‌دهد مدل چه چیزی آموخته است. چند نمونه از این خروجی‌ها را در ادامه می‌بینید.



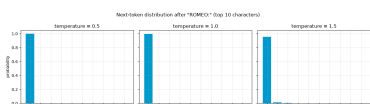
زیان، نرم‌گرادیان و توزیع وزن‌ها (قسمت چهار)



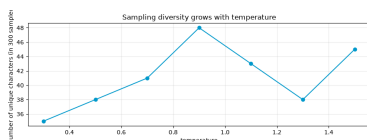
توجه آموخته‌شده؛ هر سر تخصص خود را دارد (قسمت چهار)



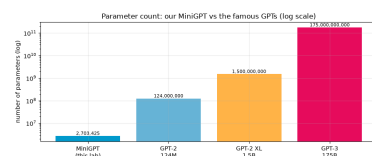
فضای امبدینگ کاراکترها با PCA (قسمت چهار)



توزیع توکن بعدی در سه دما (قسمت پنج)



تنوع نمونه‌برداری برحسب temperature (قسمت پنج)



مقیاس: از MiniGPT تا GPT-3، لگاریتمی (قسمت پنج)

منابع و دوره‌ها



NEW SHORT COURSE

How Transformer LLMs Work

Understand the transformer architecture that powers LLMs to use them more effectively

How Transformer LLMs Work ▶

یک تور تصویری از همان معماری‌ای که در این تمرین ساختید.



DeepLearning.AI

NEW SHORT COURSE

Attention in Transformers: Concepts and Code in PyTorch

Understand and implement the attention mechanism, a key element of transformer-based LLMs, using PyTorch

Attention in Transformers ▶

مکانیزم توجه به‌صورت مفهومی و کد در PyTorch.



DeepLearning.AI

Generative AI with Large Language Models

Learn the fundamentals of generative AI for real-world

Generative AI with LLMs ▶

چرخه کامل آموزش و استقرار مدل‌های بزرگ.



DeepLearning.AI

NEW COURSE BY SHARON ZHOU

Transformers in Practice

Go beyond using LLMs to actually understanding them. Build the intuition to reason about model behavior, debug real problems, and make smarter decisions about deploying transformer-based models.

Transformers in Practice ▶

کاربرد ترنسفورمرها در مسائل دنیای واقعی.



DeepLearning.AI

موفق باشید – همین مکانیزم، در مقیاس بزرگ‌تر، همان چیزی است که GPT-3 را می‌سازد.



هر سؤالی داشتید، اول Learning note های همان قسمت را ببینید.